Universitat Autònoma de Barcelona

Facultat de ciències

PRÀCTICA 1:
Recurrències numèricament estables i inestables

Grau en Matemàtiques

Guillem Gaitan Roca

2023/2024

Índex

1 Introducció i objectius **2**

2 Implementació del software **2**

2.1 Funcions trigrec 2

2.2 Escriptura de les funcions 3

2.3 Errors i temps 4

3 Manual de software **4**

3.1 Software de biblioteques i utilitats 4

3.1.1 Biblioteca `trigrec.c` 4

3.1.2 Utilitat `trigrec_escr.c` 5

3.1.3 Utilitat `trigrec_temps_err.c` 5

4 Descripció de les proves realitzades **6**

4.1 Validació del codi 6

4.2 Estudi dels temps d'execució i errors 6

4.3 Nombre d'operacions de `trigrec2()` i `trigrec2()` 7

1 Introducció i objectius

Es diu que una recurrència numèrica és inestable o mal condicionada quan l'error de cada iteració d'un algorisme es propaga als càlculs posteriors fent que el resultat final disti molt del valor real. L'objectiu d'aquesta pràctica és estudiar tres algorismes diferents per calcular sinus i cosinus i veure quin és el més estable, comparant-los amb la funció predeterminada de la llibreria `math.h`. A més s'estudiarà el temps que tarda cada algorisme en calcular un nombre `k` de funcions i el nombre d'operacions que fa cadascun per tal de determinar les operacions per segon que fa l'ordinador i les operacions necessaries per calcular les funcions sin i cos amb l'algorisme de la màquina.

L'objectiu serà, primer de tot, escriure un programa `trigrec.c` on es defineixin les funcions de quatre algorismes que calculin els valors de $\{\cos(jx)\}_{j=0}^{k}$ i $\{\sin(jx)\}_{j=0}^{k}$, i en el programa `trigrec_escr.c` s'imprimiran per pantalla. Finalment, s'ha d'escriure el programa `trigrec_temps_err.c` on s'ha d'avaluar el temps que tarda cada funció en executar-se per un cert `k` i també s'ha de calculat l'error de cada algorisme.

2 Implementació del software

Aquesta secció s'ha dividit en els tres programes que s'han desenvolupat; `trigrec.c`, `trigrec_escr.c` i `trigrec_temps_err.c`.

2.1 Funcions trigrec

La primera funció `trigrec0()` utilitza les funcions sinus i cosinus de `math.h` i les avalua en un bucle per un cert `k` i un cert `x`. Aquest algorisme és el que considerarem com a base per calcular els errors de la resta d'algorismes.

El segon algorisme consisteix en calcular les funcions trigonomètriques a partir de la següent recurrència:

$$\begin{cases} \cos((m+1)x) = 2\cos x \cos(mx) - \cos((m-1)x) \\ \sin((m+1)x) = 2\cos x \sin(mx) - \sin((m-1)x) \end{cases} \tag{1}$$

La funció `trigrec1()` és la que implementa l'algorisme 2.2[1], primer s'han hagut de definir els valors del cos i el sin per `m = 0,1`, i a partir de `m = 2` s'aplica la recurrència. En aquest cas la última iteració la farem per `m = k-1` ja que la recurrència és per `m+1`.

[1] Els algorismes als quals es fa referència estan definits en el guió de la pràctica pr01.pdf.

Per a la funció `trigrec2()` s'utilitza l'algorisme 2.3, en aquest cas la recurrència és:

$$\begin{cases} \cos((m+1)x) = \cos mx \cos x - \sin((mx) \sin x \\ \sin((m+1)x) = \sin(mx) \cos x - \cos((mx) \sin x \end{cases} \tag{2}$$

La recurrència també començarà a partir de `m = 2`, ja que cada iteració depèn dels valors anteriors i també de `m = 1`, per tant primer s'hauran de calcular per separat els sin i cos de `m = 0,1`. De la mateixa manera que en la funció `trigrec2()`, la última iteració ha de ser per `m = k-1`.

Finalment, la funció `trigrec3()` utilitza l'algorisme 2.4, la recurrència que segueix és:

$$\begin{cases} \cos((mx) = \cos((m-1)x) + \delta_m^c \\ \sin((mx) = \sin((m-1)x) + \delta_m^s \end{cases} \tag{3}$$

on definim $\delta_{m+1}^c = t\cos(mx) + \delta_m^c$ i $\delta_{m+1}^s = t\sin(mx) + \delta_m^s$. Per implementar aquest algorisme hem de definir els valors inicials:

$$\begin{aligned} \delta_1^c =& -2(\sin\frac{x}{2})^2 \\ \delta_1^s =& (-1)^{\lfloor x/\pi \rfloor}\sqrt{-\delta_1^c(2+\delta_1^c)} \\ t =& 2\delta_1^c, \cos 0 = 1, \sin 0 = 0 \end{aligned} \tag{4}$$

Per tal d'escriure $\lfloor x/\pi \rfloor$ el que s'ha fet és utilitzar la funció `floor()`, que aproxima l'expressió al valor enter inferior més pròxim. Un cop definits tots els valors per `m = 0` i `1`, comença el bucle per calcular els valors de $\cos(mx), \sin(mx), \delta_{m+1}^c$ i δ_{m+1}^s per a `m` $\geq$ `2` fins a `m = k-1`.

2.2 Escriptura de les funcions

El segon codi `trigrec_escr.c` ha d'avaluar les funcions definides en `trigrec.c` i imprimir-les per pantalla. El primer que es fa és definir una funció `resmem()` que utilitza `malloc()` per guardar la memòria, que s'utilitzarà per emmagatzemar els valors de cada funció, i comprova que efectivament l'espai es reserva correctament. Aquesta funció se li passa com a argument un `int k` i reserva la memòria per un vector de `k*sizeof(double)`. Es crida aquesta funció per cada c_i i s_i on $i = 0, 1, 2, 3$ i c fa referència al vector de cos i s al de sin. Un cop tenim tota la memòria reservada, s'avaluen les quatre funcions definides en l'apartat anterior. Finalment s'imprimeixen els valors en columnes de manera que en cada fila es llegeixen els sin i cos per un cert `m`, i s'allibera la memòria utilitzada.

2.3 Errors i temps

En l'últim codi `trigrec_temps_err.c` s'ha de calcular el temps que triga cada funcíó en fer `k` iteracions, i els errors respecte `trigrec0()`. Com en el codi anterior, primer de tot definim la funció `resmem` que actua de la mateixa manera que en l'apartat anterior i reservem la memòria per s_i i c_i. Comencem avaluant el temps que triga cada funció en executar-se, per fer-ho s'utilitza la funció `temps` que ja ens ve definida en la biblioteca `temps.c` i bàsicament indica el temps transcorregut des de l'inici de l'execució del programa. La manera d'aplicar-lo en el nostre codi és definint un `tt0` que fixa l'origen i després s'implementa la funció en el punt del codi on volem que ens fixi el temps.

Per calcula el temps d'execució de les funcións `trigrec` s'ha definit la funció `temps_funcio` que amb la funció `temps` fixa un cantador a l'inici i un cop avaluada la funció i calcula l'interval transcorregut. A més, imprimeix per pantalla els valors l'interval. Amb `temps_funcio` sabem el temps que tarda en executar-se cadascuna de les quatre funcions de trigrec.

Per avaluar els errors de `trigrec`i`()`, $i = 1, 2, 3$ respecte `trigrec0()`, es defineix la funció `maxerror()`. Aquesta funció li hem de passar com arguments els valors de les funcions sin i cos ja calculats. El seu funcionament és el següent: dins d'un bulce for es calcula, per cada `m`, la diferència en valor absolut amb la funció `fabs()` de `ci[m] - c0[m]`, on `ci[m]` és el valor de $\cos(mx)$ calculat amb l'algorisme `trigrec`i`()`. Fora del bulce s'ha inicialitzat una variable que anirà emmagatzemant els errors màxims. Dins del mateix bucle, la funció compara l'error de la m-èssima iteració amb l'emmagatzemat i en el cas que sigui més gran es guarda en la variable. Pels sin es segueix el mateix procediment.

3 Manual de software

3.1 Software de biblioteques i utilitats

En aquest apartat s'explicarà el funcionament de les funcions de les biblioteques i les utilitats:

3.1.1 Biblioteca `trigrec.c`

Les quatre funcions d'aquest codi tenen els mateixos arguments d'entrada

- `k`: valor màxim que pren `m`.
- `x`: és l'argument de $\sin(mx)$ i $\cos(mx)$.
- `c[], s[]`: guarden els valors calculats de $\sin(mx)$ i $\cos(mx)$.

i) `void trigrec0(int k, double x, double c[], double s[])`: Retorna els valors de $\sin(kx)$ i $\cos(kx)$ calculats amb les funcions `math.h` en les llistes `c[], s[]`.

ii) `void trigrec1(int k, double x, double c[], double s[])`: Retorna els valors de $\sin(kx)$ i $\cos(kx)$ calculats amb l'Algorisme 2.2 en les llistes `c[], s[]`.

iii) `void trigrec2(int k, double x, double c[], double s[])`: Retorna els valors de $\sin(kx)$ i $\cos(kx)$ calculats amb l'Algorisme 2.3 en les llistes `c[], s[]`.

iv) `void trigrec3(int k, double x, double c[], double s[])`: Retorna els valors de $\sin(kx)$ i $\cos(kx)$ calculats amb l'Algorisme 2.4 en les llistes `c[], s[]`.

3.1.2 Utilitat `trigrec_escr.c`

La utilitat `trigrec_escr.c` imprimeix per pantalla els valors de sin i cos calculats amb els quatre algorismes de `trigrec.c` per una certa `x` i `k`. Per crear l'executable `trigrec_err.o` s'introdueix la instrucció `make trigrec_escr` a la consola. Per utilitzar l'executable es fa servir la instrucció `./trigrec_escr x k`. En el fitxer `Makefile` es troben les instruccions per obtenir els executables. La funció que s'utilitza en `trigrec_escr.c` és:

i) `double * resmem(int k)`:

- Arguments d'entrada: `k`: indica que volem reservar la memòria corresponent a un vector de `k` doubles.
- Valors de retorn: reserva la memòria per guardar una llista de `k + 1` doubles i comprova que la memòria s'ha reservat correctament.

3.1.3 Utilitat `trigrec_temps_err.c`

La utilitat `trigrec_temps_err` calcula el temps d'execució dels algorismes i els errors respecte `trigrec0`. L'executable `trigrec_temps_err.o` es crea amb la instrucció `make trigrec_temps_err` i s'executa amb `./trigrec_temps_err x k`. Les funcions de la utilitat són:

i) `double maxerror(double c1[], double c2[], int k)`:

- Arguments d'entrada:
  - `c1[], c2[]`: llistes amb els valors calculats de $\sin(kx)$ i $\cos(kx)$.
  - `k`: valor màxim que pren `m`.
- Valors de retorn: Retorna, en valor absolut, l'error màxim: `max{c2[k] - c1[k]}`

ii) `void temps_funcio(void (*func)(int, double, double*, double*), int k, double x, double c[], double s[], int i)`:

- Arguments d'entrada:
  - `(*func)`: és la funció de la qual volem mesurar el temps d'execució, en aquest cas són les funcions `trigreci()`.
  - `c1[], c2[], x, k`: són els arguments que li passem a la funció `trigreci()`.
  - `k`: valor màxim que pren `m`.

- `i`: serveix per imprimir per pantalla `trigreci()` per `i = 1,2,3`

- Valors de retorn: imprimeix per pantalla l'interval de temps calculat.

4 Descripció de les proves realitzades

4.1 Validació del codi

Per comprobar que les funciones definides en la bilbioteca `trigrec.c` calculen correctament els valors de sin i cos, s'ha executat l'executable `trigrec_escr.o` amb diferents valors de `k` però amb la `x =` π`/2` fixe, ja que els valors dels sin i cos amb argument `k`π`/2` són ben coneguts.

4.2 Estudi dels temps d'execució i errors

Un dels objectius de la pràctica era avaluar el temps d'execució de les funcions de `trigrec.c`. En la Teaula 1 hi ha els intervals de temps en segons, en funció dels arguments `k` i `x`.

Taula 1: Temps, en segons, transcorregut des de l'inici fins l'execució de cada programa.

k	x	trigrec0()	trigrec1()	trigrec2()	trigrec0()
100	0.0123	$2.2 \cdot 10^{-5}$	$2.0 \cdot 10^{-6}$	$1.0 \cdot 10^{-6}$	$1.4 \cdot 10^{-6}$
1000	0,00123	$3.1 \cdot 10^{-5}$	$1.1 \cdot 10^{-5}$	$1.0 \cdot 10^{-5}$	$2.2 \cdot 10^{-5}$
10000	0,000123	$1.6 \cdot 10^{-4}$	$8.6 \cdot 10^{-5}$	$1.4 \cdot 10^{-4}$	$2.4 \cdot 10^{-4}$
100000	0,0000123	0.0013	0.0011	0.0011	0.0016
1000000	0,00000123	0.012	0.0069	0.0063	0.012
10000000	0,000000123	0.087	0.013	0.033	0.037

En la Taula 2 hem escrit els errors del càlcul dels $\cos(kx)$ i en la Taula 3 els del $\sin(kx)$, tots en funció de `k` i `x`. En el cas de la funció `trigrec1()` veiem que l'error en els dos casos incrementa en dos ordres de magnitud cada cop que augmentem un ordre en `k`, per tant veiem que, tal i com predia l'estudi de l'error del guió, la recurrència de l'Algorisme 2.2 és numèricament intestable.

Taula 2: Errors comesos per les funcions `trigreci()` per $i = 1, 2, 3$ respecte `trigrec0()` en el càlcul del cos.

k	x	Δ_c^1	Δ_c^2	Δ_c^3
100	0.0123	$2.253 \cdot 10^{-13}$	$1.332 \cdot 10^{-15}$	$3.331 \cdot 10^{-16}$
1000	0,00123	$2.063 \cdot 10^{-11}$	$1.266 \cdot 10^{-14}$	$1.332 \cdot 10^{-15}$
10000	0,000123	$2.751 \cdot 10^{-9}$	$1.674 \cdot 10^{-13}$	$4.108 \cdot 10^{-15}$
100000	0,0000123	$3.517 \cdot 10^{-7}$	$2.086 \cdot 10^{-12}$	$7.661 \cdot 10^{-15}$
1000000	0,00000123	$4.204 \cdot 10^{-5}$	$2.504 \cdot 10^{-11}$	$2.387 \cdot 10^{-14}$
10000000	0,000000123	$1.009 \cdot 10^{-3}$	$6.241 \cdot 10^{-11}$	$1.188 \cdot 10^{-13}$

En el cas de l'algorisme 2.3, també veiem que per cada ordre de magnitud que augmentem `k`, l'error també n'augmenta un, tot i això és més precís que l'algorisme 2.2. Finalment, l'últim algorisme, és el més estable de tots, malgrat que al final sí que va augmentant l'error, veiem que l'augment no és lineal amb la `k`. Comparant amb els altres algorismes l'error per a `k` grans, és 10 ordres de maginud més petit que l'error de `trigrec1`, i 3 que el de `trigrec2`.

Taula 3: Errors comesos per les funcions `trigreci()` per $i = 1, 2, 3$ respecte `trigrec0()` en el càlcul del sin.

k	x	Δ_s^1	Δ_s^2	Δ_s^3
100	0.0123	$1.271 \cdot 10^{-13}$	$2.887 \cdot 10^{-15}$	$2.220 \cdot 10^{-16}$
1000	0,00123	$1.034 \cdot 10^{-11}$	$2.576 \cdot 10^{-14}$	$7.772 \cdot 10^{-16}$
10000	0,000123	$1.129 \cdot 10^{-9}$	$3.374 \cdot 10^{-13}$	$5.662 \cdot 10^{-15}$
100000	0,0000123	$1.615 \cdot 10^{-7}$	$4.328 \cdot 10^{-12}$	$9.236 \cdot 10^{-15}$
1000000	0,00000123	$1.933 \cdot 10^{-5}$	$5.182 \cdot 10^{-11}$	$4.696 \cdot 10^{-14}$
10000000	0,000000123	$5.332 \cdot 10^{-4}$	$1.322 \cdot 10^{-10}$	$7.583 \cdot 10^{-14}$

4.3 Nombre d'operacions de `trigrec2()` i `trigrec2()`

A continuació anem a contar el número d'operacions que fan les funcions `trigrec2()` i `trigrec3()` en funció de `k`, només trindrem en compte les operacions de dintre els bucles, ja que per `k` grans, la resta es poden menysprear:

i) `trigrec2()`: Aquesta funció fa quatre multiplicacions, una suma i una resta per calcular cada iteració de `c[m+1], s[m+1]`. Com que el bucle s'inicia per `m=1` i finalitza en `m = k-1`, el nombre d'operacions serà `6(k-1)`.

ii) `trigrec3()`: En aquest cas les operacions que fa per cada iteració són: 4 sumes i 2 multiplicacions. El bucle s'inicia per `m = 1` fins a `m = k`. Per tant el nombre d'operacions dins del bucle és `6k`.

Utilitzant el temps obtinguts en la Taula 1 dels dos últims algorismes s'ha determinat que el nombre d'operacions que pot realitzar l'ordinador `ASUS Zenbook UX3402VA_UX3402VA` és de l'ordre d'entre $5 \cdot 10^8$ i 10^9 operacions per segon, que és l'estàndard en ordinadors portàtils. Els valors dels càlculs realitzats estan en la Taula 4 i la Taula 5.

Taula 4: Temps, en segons, transcorregut des de l'inici fins l'execució de cada programa.

k	100	1000	10000	100000	1000000	10000000
temps (s)	$1.0 \cdot 10^{-6}$	$1.0 \cdot 10^{-5}$	$1.4 \cdot 10^{-4}$	0.0011	0.0063	0.033
nombre d'operacions	594	5994	59994	599994	5999994	59999994
Operacions/s	$5.9 \cdot 10^8$	$6.0 \cdot 10^8$	$4.3 \cdot 10^8$	$5.5 \cdot 10^8$	$9.5 \cdot 10^8$	$1.9 \cdot 10^9$

Taula 5: Temps, en segons, transcorregut des de l'inici fins l'execució de cada programa.

`k`	100	1000	10000	100000	1000000	10000000
`temps (s)`	$1.4 \cdot 10^{-6}$	$2.2 \cdot 10^{-5}$	$2.4 \cdot 10^{-4}$	0.0016	0.012	0.037
`nombre d'operacions`	600	6000	60000	600000	6000000	60000000
`Operacions/s`	$4.3 \cdot 10^{8}$	$2.7 \cdot 10^{8}$	$2.5 \cdot 10^{8}$	$3.8 \cdot 10^{8}$	$5.0 \cdot 10^{8}$	$1.6 \cdot 10^{9}$

Finalment, sabent les operacions que pot fer l'ordinador per segon, podem estimar el nombre d'operacions que equival calcular un sin o un cos amb l'algorisme 1, és a dir, la funció de la biblioteca `math.h`. Hem de considerar que dins de la funció es calculen `k` sin i `k` cos, per tant en total es calculen `2k` funcions trigonomètriques. És una estimació ja que treballem amb ordres de temps molt petits i la variància d'aquests resultats és gran. El nombre d'operacions que fa la màquina per calcular un sin o un cos és aproximadament 6, altre cop recordem que és una estimació i que possiblement aquest valor disti molt del valor real.